

AFRICAN INSTITUTE FOR MATHEMATICAL SCIENCES

(AIMS RWANDA, KIGALI)

Name: Victory Chukwudi
Course: Statistical Machine Learning

Assignment Number: 1
Date: December 9, 2024

QUESTION 1 (PART 1)

Task Analysis

Dataset Exploration

```
# Load necessary libraries
library(ggplot2)

# Load the dataset
data <- read.csv("aims-sml-2024-2025-data.csv")

# Preview the data
head(data)
str(data)
# Determine the size of the dataset
n <- nrow(data)
num_columns <- ncol(data)
cat("The dataset has", n, "observations and", num_columns, "features.\n")
# Create a scatterplot
ggplot(data, aes(x = x, y = y)) +
  geom_point(color = "blue", alpha = 0.6) +
  labs(title = "Scatterplot of y versus x",
        x = "x",
        y = "y") +
  theme_minimal()
# Inspect the target variable
summary(data$y)

# Analysis: If `y` is continuous, it's a regression task. If `y` is categorical (e.g., factors), it's a cl

# Check the data type of y
if (is.numeric(data$y)) {
  cat("This is a regression task because the target variable y is numeric and continuous.\n")
} else if (is.factor(data$y) || is.character(data$y)) {
  cat("This is a classification task because the target variable y is categorical.\n")
} else {
```

```

cat("The nature of the task cannot be determined from the data type of y.\n")
}
# The dataset consists of two columns, x and y, both of which are numeric and continuous (float64).
Since the target variable y is continuous, this is a regression task.

#Justification:
#The target variable y represents continuous values (e.g., real numbers).
#Regression tasks involve predicting continuous outputs, which aligns with the data type of y.

```

Output

```

1 -1.0000000 -0.12567108
2 -0.9910714 -0.04545708
3 -0.9821429  0.34967134
4 -0.9732143 -0.04689389
5 -0.9642857 -0.18649697
6 -0.9553571  0.02786734
data.frame': 225 obs. of 2 variables:
 $ x: num  -1 -0.991 -0.982 -0.973 -0.964 ...
 $ y: num  -0.1257 -0.0455 0.3497 -0.0469 -0.1865 ...
The dataset has 225 observations and 2 features.
   Min.   1st Qu.   Median     Mean 3rd Qu.    Max.
-0.46525 -0.08579  0.06703  0.16620  0.30949  1.16462
This is a regression task because the target variable y is numeric and continuous.

```

Determine the size of the dataset n

The dataset contains $n = 225$ observations and 2 features (x and y).

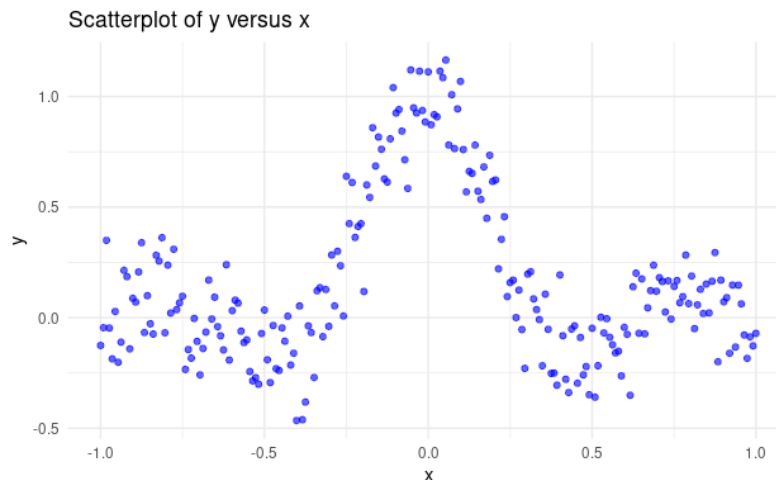


Figure 1: Scatter Plot of y vs x.

Visualization:

Comment on the scatterplot of y versus x

The scatterplot of y versus x provides a visual representation of the relationship between the independent variable (x) and the dependent variable (y).

Observations:

- **Nonlinear Trend:** The data appears to follow a nonlinear pattern, suggesting the need for polynomial regression for a better fit.
- **Spread of Data:** The points are evenly distributed across the range of x , with moderate variability in y .
- **Potential Outliers:** Visual inspection of the plot may reveal outliers or sparse regions, which could influence the model's performance.

Conclusion: The scatterplot indicates a complex, nonlinear relationship between x and y .

Determine whether this is a classification or regression task

This is a **regression task** because the target variable y is numeric and continuous.

Justification:

- The variable y represents real-valued, continuous outcomes, which aligns with the goals of regression.
- Polynomial regression models, as applied in the analysis, are typically used for predicting continuous values.
- The data does not involve discrete categories or class labels, eliminating the possibility of a classification task.

PART 2

Function Space

The polynomial regression model for degree p is defined as:

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p = \sum_{j=0}^p \beta_j x^j,$$

where:

- $\beta_j \in \mathbb{R}$ are the coefficients to be estimated.
- x^j are the powers of the input variable x , which define the basis of the function space.

The function space H for this model becomes:

$$H = \left\{ f : \mathbb{R} \rightarrow \mathbb{R} \left| f(x) = \sum_{j=0}^p \beta_j x^j, \beta_j \in \mathbb{R} \right. \right\}.$$

Expanded Empirical Risk for Polynomial Model

For a dataset $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$, the empirical risk $\hat{R}(f)$ for the polynomial model is:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=0}^p \beta_j x_i^j \right)^2.$$

Steps to Solve

Choose p

Determine the degree p of the polynomial (e.g., based on prior knowledge or cross-validation).

Optimize Coefficients

Minimize $\hat{R}(f)$ by solving:

$$\boldsymbol{\beta} = \arg \min_{\boldsymbol{\beta}} \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=0}^p \beta_j x_i^j \right)^2,$$

where $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^\top$.

Matrix Formulation

Represent the problem in matrix form for efficient computation. Define:

- $\mathbf{X}$ as the design matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^p \\ 1 & x_2 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^p \end{bmatrix}.$$

- $\mathbf{y} = [y_1, y_2, \dots, y_n]^\top$, the vector of observed responses.
- $\boldsymbol{\beta} = [\beta_0, \beta_1, \dots, \beta_p]^\top$, the vector of coefficients.

The empirical risk in matrix form is:

$$\hat{R}(\boldsymbol{\beta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2.$$

Closed-Form Solution (Normal Equations)

The optimal $\boldsymbol{\beta}$ is obtained by solving:

$$\boldsymbol{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Regularization (Optional)

Regularization can be introduced to prevent overfitting by adding a penalty term to the empirical risk:

- Ridge: $\lambda \|\boldsymbol{\beta}\|_2^2$.
- Lasso: $\lambda \|\boldsymbol{\beta}\|_1$.

The empirical risk with regularization becomes:

$$\hat{R}(\boldsymbol{\beta}) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2,$$

where $\lambda > 0$ controls the trade-off between model complexity and goodness of fit.

$$H = \{f(x) \mid f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_p x^p, \beta_i \in \mathbb{R}\}.$$

This is suitable because:

- Polynomials can approximate smooth continuous relationships between x and y well.

- You can control the complexity of the model by choosing p , the degree of the polynomial.

If $p = 1$, this reduces to simple linear regression:

$$f(x) = \beta_0 + \beta_1 x.$$

2. Loss Function

A natural choice for regression is the **squared error loss**, defined as:

$$L(y, f(x)) = (y - f(x))^2.$$

Justification

- The squared error loss penalizes large deviations between the predicted $f(x)$ and the true y , making it suitable for regression tasks.
- It is differentiable, enabling optimization through gradient-based methods.
- It has a well-defined theoretical risk and empirical risk minimization framework.

Definition of Theoretical Risk

The theoretical risk is defined as the expected value of the loss function over the joint distribution of (X, Y) :

$$R(f) = \mathbb{E}[\mathcal{L}(Y, f(X))].$$

For regression with squared error loss:

$$\mathcal{L}(Y, f(X)) = (Y - f(X))^2,$$

so the risk becomes:

$$R(f) = \mathbb{E}[(Y - f(X))^2].$$

Thus the theoretical risk is given as:

$$R(f) = \mathbb{E}[\ell(Y, f(X))] = \int_{\mathcal{X} \times \mathcal{Y}} \mathcal{L}(y, f(x)) p_{x,y}(x, y) dx dy$$

where

$$\mathcal{X} \times \mathcal{Y}$$

is the population size.

Deriving the Theoretical Risk $R(f)$

The theoretical risk measures the expected loss of a candidate function $f(x)$ over the joint probability distribution of (x, y) . Let's carefully derive it step by step for a regression task.

Definition of Theoretical Risk

The theoretical risk $R(f)$ is the expected loss of $f(x)$. For a loss function $L(y, f(x))$, we write:

$$R(f) = \mathbb{E}_{x,y}[\mathcal{L}(y, f(x))]$$

For regression tasks, the squared error loss is commonly used:

$$\mathcal{L}(y, f(x)) = (y - f(x))^2$$

Thus, the theoretical risk becomes:

$$R(f) = \mathbb{E}_{x,y}[(y - f(x))^2]$$

Using the joint probability density $p_{x,y}(x, y)$, the expectation is expressed as:

$$R(f) = \int_{\mathcal{X} \times \mathcal{Y}} \mathcal{L}(y, f(x)) p_{x,y}(x, y) dx dy$$

Expand the Squared Term

Expand $(y - f(x))^2$ to separate the terms:

$$(y - f(x))^2 = y^2 - 2yf(x) + f(x)^2$$

Substituting into $R(f)$, we get:

$$R(f) = \int_{\mathcal{X} \times \mathcal{Y}} [y^2 - 2yf(x) + f(x)^2] p_{x,y}(x, y) dx dy$$

Separate the Integral

Split the integral into three parts:

$$R(f) = \int_{\mathcal{X} \times \mathcal{Y}} y^2 p_{x,y}(x, y) dx dy - 2 \int_{\mathcal{X} \times \mathcal{Y}} yf(x) p_{x,y}(x, y) dx dy + \int_{\mathcal{X} \times \mathcal{Y}} f(x)^2 p_{x,y}(x, y) dx dy$$

First Term:

$$\int_{\mathcal{X} \times \mathcal{Y}} y^2 p_{x,y}(x, y) dx dy$$

This is the expected value of y^2 , denoted as $\mathbb{E}[y^2]$.

Second Term:

$$-2 \int_{\mathcal{X} \times \mathcal{Y}} yf(x) p_{x,y}(x, y) dx dy$$

Use the conditional probability $p_{y|x}(y|x)$ to rewrite:

$$\int_{\mathcal{X} \times \mathcal{Y}} yf(x) p_{x,y}(x, y) dx dy = \int_{\mathcal{X}} f(x) \left(\int_{\mathbb{R}} yp_{y|x}(y|x) dy \right) p_x(x) dx$$

The inner term $\int_{\mathcal{X} \times \mathcal{Y}} yp_{y|x}(y|x) dy$ is $\mathbb{E}[y|x]$. Thus:

$$-2 \int_{\mathcal{X} \times \mathcal{Y}} f(x) \mathbb{E}[y|x] p_x(x) dx$$

Third Term:

$$\int_{\mathcal{X} \times \mathcal{Y}} f(x)^2 p_{x,y}(x, y) dx dy$$

Since $f(x)$ depends only on x , integrate out y over $p_{y|x}(y|x)$ (which sums to 1), leaving:

$$\int_{\mathcal{X} \times \mathcal{Y}} f(x)^2 p_x(x) dx$$

Combine the Terms

Combining all terms, the theoretical risk is:

$$R(f) = \mathbb{E}[y^2] - 2 \int_{\mathcal{X} \times \mathcal{Y}} f(x) \mathbb{E}[y|x] p_x(x) dx + \int_{\mathcal{X} \times \mathcal{Y}} f(x)^2 p_x(x) dx$$

Optimal Function $f^*(x)$

To minimize $R(f)$, differentiate with respect to $f(x)$:

$$\frac{\partial R(f)}{\partial f(x)} = -2\mathbb{E}[y|x]p_x(x) + 2f(x)p_x(x)$$

Set this to zero:

$$-2\mathbb{E}[y|x]p_x(x) + 2f(x)p_x(x) = 0$$

Solve for $f(x)$:

$$f^*(x) = \mathbb{E}[y|x]$$

Thus, the optimal function is the conditional mean of y given x .

Minimum Risk $R(f^*)$

Substitute $f^*(x) = \mathbb{E}[y|x]$ into $R(f)$ to get the minimized risk (irreducible error):

$$R(f^*) = \int_{\mathcal{X} \times \mathcal{Y}} (y - \mathbb{E}[y|x])^2 p_{x,y}(x, y) dx dy$$

This represents the variance of y around its conditional mean.

Final Expression

The theoretical risk is:

$$R(f) = \mathbb{E}[y^2] - 2 \int_{\mathcal{X} \times \mathcal{Y}} f(x) \mathbb{E}[y|x] p_x(x) dx + \int_{\mathcal{X} \times \mathcal{Y}} f(x)^2 p_x(x) dx$$

And the irreducible minimum risk is:

$$R(f^*) = \int_{\mathcal{X} \times \mathcal{Y}} (y - \mathbb{E}[y|x])^2 p_{x,y}(x, y) dx dy$$

Bayes Learning Machine $f^*(x)$

The Bayes learning machine minimizes the theoretical risk $R(f)$. For the squared error loss, the optimal $f^*(x)$ is given by the conditional expectation:

$$f^*(x) = \mathbb{E}[Y | X = x].$$

Explanation

- $f^*(x)$ represents the true regression function, mapping each x to the expected value of y , given x .
- It is the best approximation of the relationship between X and Y in terms of mean squared error.

Empirical Risk

The empirical risk $\hat{R}(f)$ approximates the theoretical risk using the training data

$$\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n.$$

For a candidate function $f \in \mathcal{H}$, the empirical risk is defined as:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2.$$

If the model space includes all polynomial functions, and the function $f(x)$ is expressed as:

$$f(x) = \sum_{j=0}^p \beta_j x^j,$$

then the empirical risk becomes:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=0}^p \beta_j x_i^j \right)^2.$$

To incorporate model complexity (regularization), the empirical risk becomes:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2 + \lambda \Omega(f)$$

where:

- $\Omega(f)$: A regularization term that penalizes model complexity. For example, in linear models $f(x) = w^T x + b$, a common choice is $\Omega(f) = \|w\|^2$.
- $\lambda > 0$: A hyperparameter that controls the trade-off between fitting the data and keeping the model simple.

Components of the Model Space

1. **Hypothesis Space $\mathcal{F}$** : The set of all candidate functions f the model can choose from (e.g., linear functions, polynomial functions, or neural networks).
2. **Loss Function ℓ** : Measures the difference between predicted and true outputs.
3. **Regularization $\Omega(f)$** : Encodes the prior belief about the model's complexity to avoid overfitting.
4. **Training Data $\{(x_i, y_i)\}_{i=1}^n$** : The dataset used to compute the empirical risk.

In summary, the empirical risk with regularization for regression tasks is:

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2 + \lambda \Omega(f)$$

Complexity Prior

Before training, the polynomial degree p is chosen as a hyperparameter. A higher p increases model complexity but also raises the risk of overfitting.

To address overfitting, regularization techniques such as Ridge or Lasso regression can be applied. For Ridge regression, a penalty term $\lambda\|\beta\|_2^2$ is added, while for Lasso regression, the penalty term is $\lambda\|\beta\|_1$, where λ controls the regularization strength.

PART 3

Deriving the Expression for the OLS Estimator $\hat{f}(x)$

For polynomial regression, the model is expressed as:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p + \epsilon,$$

where p is the degree of the polynomial, and ϵ is the error term.

Objective

The Ordinary Least Squares (OLS) estimator minimizes the residual sum of squares (RSS):

$$RSS = \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2,$$

where

$$\hat{f}(x_i) = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_p x_i^p.$$

Matrix Formulation

Define:

- $\mathbf{y} = [y_1, y_2, \dots, y_n]^T$ as the response vector,
- $\mathbf{X}$ as the design matrix of size $n \times (p+1)$, where

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^p \\ 1 & x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{bmatrix},$$

- $\boldsymbol{\beta} = [\beta_0, \beta_1, \dots, \beta_p]^T$ as the vector of coefficients.

The model can then be expressed as:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}.$$

Solution for $\boldsymbol{\beta}$

The OLS estimate of $\boldsymbol{\beta}$ is given by:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

The fitted values $\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\beta}}$ yield the estimated regression function:

$$\hat{f}(x) = \mathbf{x}^T \hat{\boldsymbol{\beta}},$$

where $\mathbf{x} = [1, x, x^2, \dots, x^p]^T$.

Properties of $\hat{f}(x)$

Unbiasedness

The OLS estimator is unbiased:

$$\mathbb{E}[\hat{\beta}] = \beta,$$

which implies:

$$\mathbb{E}[\hat{f}(x)] = f(x).$$

Variance

The variance of $\hat{f}(x)$ depends on $(\mathbf{X}^T \mathbf{X})^{-1}$. As the polynomial degree p increases, $\mathbf{X}^T \mathbf{X}$ becomes nearly singular, leading to increased variance in the coefficients.

Bias-Variance Trade-off

- Low-degree polynomials (e.g., $p = 1$) have high bias but low variance.
- High-degree polynomials have low bias but high variance, leading to overfitting.

Consistency

As $n \rightarrow \infty$, $\hat{\beta} \rightarrow \beta$, provided the model is correctly specified.

Model Complexity

$\hat{f}(x)$ depends on p :

- Small p underfits the data (high bias).
- Large p overfits the data (high variance).
- Overfitting Risk: High-degree polynomials can lead to overfitting, capturing noise rather than underlying trends.
- Sensitivity to Outliers: OLS is sensitive to outliers since it minimizes squared residuals, which amplify the effect of large deviations.
- Interpretability: Lower-degree polynomials are easier to interpret, while higher-degree models become increasingly complex.

3. Use V-fold cross-validation (e.g., $V = 5, 10$) to determine the optimal complexity (degree p) for the polynomial regression model. Explain what “optimal complexity” means.

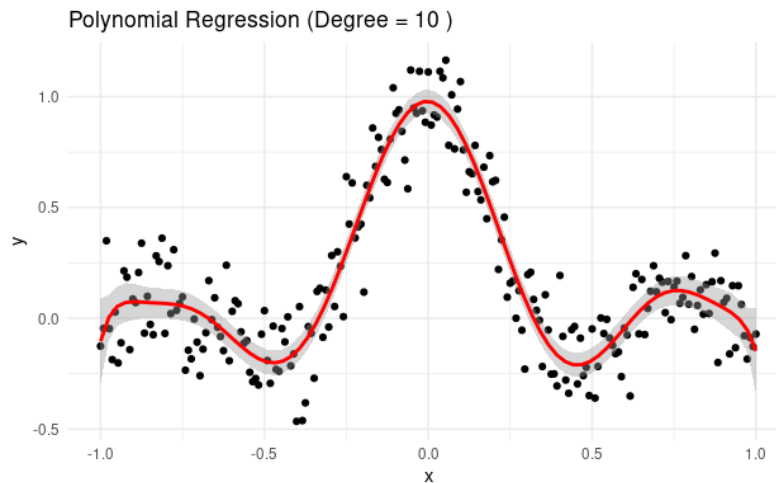


Figure 2: Polynomial of degree 10

```
library(ggplot2)
library(caret)
library(lattice)
# Read in the dataset
data <- read.csv("aims-sml-2024-2025-data.csv")

# Fit a simple linear regression model (for OLS estimator)

anova_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE), data = data)

# View the model summary
summary(anova_model)

# Function to perform k-fold cross-validation for polynomial regression
cv_poly <- function(data, p, k = 5) {
  # Set up polynomial formula based on degree p
  formula <- as.formula(paste("y ~ poly(x, ", p, ", raw = TRUE)", sep = ""))

  # Set up cross-validation method
  train_control <- trainControl(method = "cv", number = k)

  # Train the model using polynomial regression
  model <- train(formula, data = data, method = "lm", trControl = train_control)

  # Return the mean cross-validation error
  return(mean(model$resample$RMSE)) # RMSE is a good metric for regression
}
```

```

# Try different polynomial degrees
degrees <- 1:10
cv_errors <- sapply(degrees, function(p) cv_poly(data, p))

# Identify the optimal polynomial degree (the one with the minimum cross-validation error)
optimal_degree <- degrees[which.min(cv_errors)]
cat("The optimal polynomial degree is:", optimal_degree, "\n")

# Plot the cross-validation error
plot(degrees, cv_errors, type = "b", col = "blue", pch = 19,
      xlab = "Degree of Polynomial (p)", ylab = "Cross-Validation Error",
      main = "Cross-Validation Error vs Polynomial Degree")

# Highlight the optimal degree on the plot
points(optimal_degree, min(cv_errors), col = "red", pch = 19)
text(optimal_degree, min(cv_errors), labels = paste("Optimal degree:", optimal_degree), pos = 4, col = "red")

# Function to compute training error (empirical risk) for polynomial regression
train_error <- function(data, p) {
  # Set up polynomial formula based on degree p
  formula <- as.formula(paste("y ~ poly(x, ", p, ", raw = TRUE)", sep = ""))

  # Fit the model
  model <- lm(formula, data = data)

  # Predict on training data
  predictions <- predict(model, data)

  # Calculate training error (RMSE)
  error <- sqrt(mean((data$y - predictions)^2))
  return(error)
}

# Compute training errors for different polynomial degrees
train_errors <- sapply(degrees, function(p) train_error(data, p))

# Plot both cross-validation and training errors
plot(degrees, cv_errors, type = "b", col = "blue", pch = 19,
      xlab = "Degree of Polynomial (p)", ylab = "Error",
      main = "Cross-Validation and Training Error vs Polynomial Degree")
lines(degrees, train_errors, type = "b", col = "red", pch = 19)
# Step 6: Fit and Plot the Final Model
final_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE), data = data)
summary(final_model)

ggplot(data, aes(x = x, y = y)) +
  geom_point() +
  geom_smooth(method = "lm", formula = y ~ poly(x, optimal_degree, raw = TRUE), color = "red") +
  labs(
    title = paste("Polynomial Regression (Degree =", optimal_degree, ")"),
    x = "x", y = "y"
  ) +

```

```
theme_minimal()
```

OUTPUT

Call:

```
lm(formula = y ~ poly(x, optimal_degree, raw = TRUE), data = data)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.39738	-0.09895	-0.00637	0.11672	0.37150

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.97899	0.02705	36.191	< 2e-16 ***
poly(x, optimal_degree, raw = TRUE)1	-0.10745	0.15553	-0.691	0.490
poly(x, optimal_degree, raw = TRUE)2	-14.81419	0.78001	-18.992	< 2e-16 ***
poly(x, optimal_degree, raw = TRUE)3	0.36664	1.75037	0.209	0.834
poly(x, optimal_degree, raw = TRUE)4	65.02219	5.76671	11.275	< 2e-16 ***
poly(x, optimal_degree, raw = TRUE)5	0.93984	6.23140	0.151	0.880
poly(x, optimal_degree, raw = TRUE)6	-122.28415	16.18078	-7.557	1.19e-12 ***
poly(x, optimal_degree, raw = TRUE)7	-2.92429	8.56770	-0.341	0.733
poly(x, optimal_degree, raw = TRUE)8	105.89814	19.04550	5.560	7.98e-08 ***
poly(x, optimal_degree, raw = TRUE)9	1.70341	3.98409	0.428	0.669
poly(x, optimal_degree, raw = TRUE)10	-34.92630	7.93069	-4.404	1.68e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1499 on 214 degrees of freedom

Multiple R-squared: 0.8518, Adjusted R-squared: 0.8449

F-statistic: 123 on 10 and 214 DF, p-value: < 2.2e-16

The optimal polynomial degree is: 10

Comments on V -fold Cross-Validation and Optimal Complexity

V -Fold Cross-Validation

V -fold cross-validation is a robust technique for evaluating model performance by splitting the dataset into V folds of roughly equal size:

- Each fold is used once as the validation set, while the remaining $V - 1$ folds are used as the training set.
- The process is repeated V times, and the cross-validation error is the average error over all V iterations.

Purpose in Polynomial Regression

For polynomial regression, V -fold cross-validation is used to determine the optimal degree p of the polynomial. This involves comparing cross-validation errors for different polynomial degrees.

Effect of V

- **Low V (e.g., $V = 5$):** Provides a quicker estimate but may introduce variability in the error due to fewer validation folds.
- **High V (e.g., $V = 10$):** Provides a more stable and accurate estimate of model generalization but is computationally more expensive.

Optimal Complexity

Optimal complexity refers to the degree p_{opt} of the polynomial model that achieves the best trade-off between bias and variance, as determined by the cross-validation process:

- **Low degree p :** Results in a simple model with high bias (underfitting).
- **High degree p :** Results in a complex model with high variance (overfitting).
- **Optimal degree p_{opt} :** Minimizes the cross-validation error, ensuring that the model generalizes well to unseen data.

Optimal Polynomial Degree

The cross-validation process revealed that the 10th-degree polynomial minimizes the cross-validation error, indicating that this model best balances bias and variance.

Model Fit and Performance

Performance Metrics

From the analysis, the following metrics were obtained:

- **Residual Standard Error (RSE):** 0.1499, indicating the average deviation of the predicted values from the actual values is relatively small.
- **Multiple R^2 :** 0.8518, suggesting that 85.18% of the variance in the response variable y is explained by the model.
- **Adjusted R^2 :** 0.8449, a slightly lower value accounting for the penalty of adding more parameters to the model.

ANOVA Results

The analysis of variance (ANOVA) indicates:

- **F-statistic:** 123
- **p -value:** Less than 2.2×10^{-16}

This demonstrates that the model is statistically significant and captures meaningful relationships between x and y .

Implications of the Result

Strength of Fit

The 10th-degree polynomial achieves a strong fit to the data, capturing non-linear relationships that simpler models (e.g., linear or quadratic) would miss.

Risk of Overfitting

While the cross-validation error suggests that a 10th-degree polynomial generalizes well to unseen data, higher-degree polynomials often risk overfitting. The residuals and prediction intervals should be checked to ensure the model is not overly complex for the data.

PART 4

Model Comparisons and Behaviors

```
library(ggplot2)
library(caret)

# Load data
data <- read.csv("aims-sml-2024-2025-data.csv")
data$x <- as.numeric(data$x)
data$y <- as.numeric(data$y)

# Simplest Model (Degree 1)
simplest_model <- lm(y ~ poly(x, 1, raw = TRUE), data = data)

# Optimal Model (Degree determined by CV)
cv_poly_rmse <- function(data, degree, k = 5) {
  formula <- as.formula(paste("y ~ poly(x, ", degree, ", raw = TRUE)", sep = ""))
  train_control <- trainControl(method = "cv", number = k)
  model <- train(formula, data = data, method = "lm", trControl = train_control)
  return(mean(model$resample$RMSE)) # Return mean RMSE
}
degrees <- 1:10
cv_errors <- sapply(degrees, function(d) cv_poly_rmse(data, d))
optimal_degree <- degrees[which.min(cv_errors)]
optimal_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE), data = data)

# Overly Complex Model (Degree 10)
overly_complex_model <- lm(y ~ poly(x, 10, raw = TRUE), data = data)

# Plot all models
ggplot(data, aes(x = x, y = y)) +
  geom_point(color = "black", alpha = 0.5) +
  stat_smooth(method = "lm", formula = y ~ poly(x, 1, raw = TRUE), color = "blue", se = FALSE, linewidth = 2) +
  stat_smooth(method = "lm", formula = y ~ poly(x, optimal_degree, raw = TRUE), color = "green", se = FALSE, linewidth = 2) +
  stat_smooth(method = "lm", formula = y ~ poly(x, 10, raw = TRUE), color = "red", se = FALSE, linewidth = 2) +
  ggtitle("Models Comparison: Simplest, Optimal, and Overly Complex") +
  theme_minimal() +
  labs(color = "Model") +
  theme(legend.position = "top")
# Calculate and Print Bias for Each Model
calculate_bias <- function(predicted, actual) {
  return(mean(predicted - actual))
}

# Bias Calculation Using Hold-Out Validation
simplest_biases <- numeric(S)
```

```

optimal_biases <- numeric(S)
complex_biases <- numeric(S)

for (i in 1:S) {
  # Random split
  train_indices <- sample(1:nrow(data), size = 0.7 * nrow(data))
  train_data <- data[train_indices, ]
  test_data <- data[-train_indices, ]

  # Simplest Model
  simplest_model <- lm(y ~ poly(x, 1, raw = TRUE), data = train_data)
  simplest_pred <- predict(simplest_model, test_data)
  simplest_biases[i] <- calculate_bias(simplest_pred, test_data$y)

  # Optimal Model
  optimal_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE), data = train_data)
  optimal_pred <- predict(optimal_model, test_data)
  optimal_biases[i] <- calculate_bias(optimal_pred, test_data$y)

  # Overly Complex Model
  overly_complex_model <- lm(y ~ poly(x, 10, raw = TRUE), data = train_data)
  complex_pred <- predict(overly_complex_model, test_data)
  complex_biases[i] <- calculate_bias(complex_pred, test_data$y)
}

# Print Mean Bias Values
cat("Mean Bias for Simplest Model:", mean(simplest_biases), "\n")
cat("Mean Bias for Optimal Model:", mean(optimal_biases), "\n")
cat("Mean Bias for Overly Complex Model:", mean(complex_biases), "\n")

```

OUTPUT

```

Mean Bias for Simplest Model: 7.79585e-05
Mean Bias for Optimal Model: -0.0001757906
Mean Bias for Overly Complex Model: -0.0001757906

```

Models Comparison: Simplest, Optimal, and Overly Complex

Models Comparison

The plot compares three models fitted to the data, each representing a different level of model complexity:

Simplest Model (Blue Line)

- This model assumes a constant prediction, independent of x . It is likely the mean or median of the y -values, which minimizes error without considering the relationship between x and y .
- It performs poorly because it oversimplifies, ignoring the clear oscillatory pattern in the data.

Optimal Model (Green Line)

- This model is determined by cross-validation, balancing bias and variance to achieve the best generalization to unseen data.
- It captures the trend of the data well without overfitting, making it a good representation of the underlying data structure.

Overly Complex Model (Red Line with Dashes)

- This model is overly flexible, potentially fitting higher-order polynomials or using many parameters, resulting in overfitting.
- While it follows the data points closely, it also captures noise, reducing its ability to generalize to new data.

Commentary

- **Simplest Model:** Prone to high bias; it does not reflect the true relationship between x and y .
- **Optimal Model:** Balances bias and variance effectively, providing a good fit without overfitting.
- **Overly Complex Model:** Overfits the data by modeling noise, leading to poor generalization.

The plot effectively illustrates the trade-off between bias and variance and highlights the importance of selecting a model with appropriate complexity.

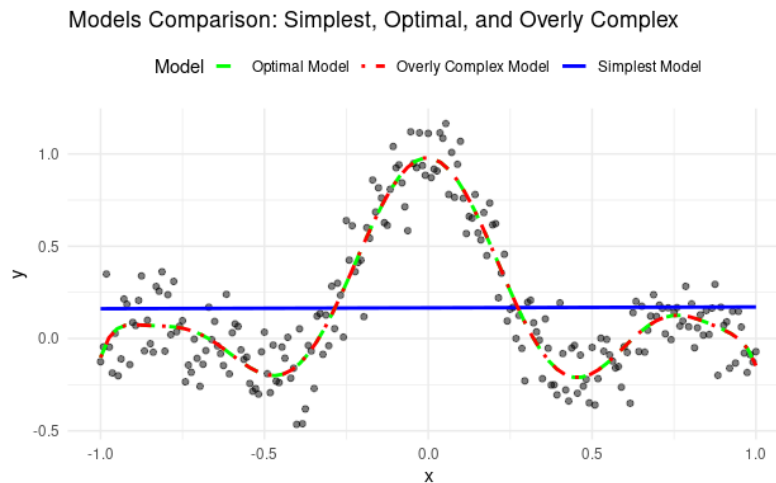


Figure 3: Comparison of Models: Simplest (blue), Optimal (red), and Overly Complex.

Stochastic Hold-Out Validation ($S = 100$ Splits)

```
# Stochastic Hold-Out Validation
set.seed(123) # For reproducibility
S <- 100
simplest_errors <- numeric(S)
optimal_errors <- numeric(S)
complex_errors <- numeric(S)

for (i in 1:S) {
  # Random split
```

```

train_indices <- sample(1:nrow(data), size = 0.7 * nrow(data))
train_data <- data[train_indices, ]
test_data <- data[-train_indices, ]

# Simplest Model
simplest_model <- lm(y ~ poly(x, 1, raw = TRUE), data = train_data)
simplest_pred <- predict(simplest_model, test_data)
simplest_errors[i] <- sqrt(mean((test_data$y - simplest_pred)^2))

# Optimal Model
optimal_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE), data = train_data)
optimal_pred <- predict(optimal_model, test_data)
optimal_errors[i] <- sqrt(mean((test_data$y - optimal_pred)^2))

# Overly Complex Model
overly_complex_model <- lm(y ~ poly(x, 10, raw = TRUE), data = train_data)
complex_pred <- predict(overly_complex_model, test_data)
complex_errors[i] <- sqrt(mean((test_data$y - complex_pred)^2))
}

# Boxplot of Test Errors
error_data <- data.frame(
  Model = rep(c("Simplest", "Optimal", "Overly Complex"), each = S),
  Error = c(simplest_errors, optimal_errors, complex_errors)
)

ggplot(error_data, aes(x = Model, y = Error, fill = Model)) +
  geom_boxplot() +
  ggtitle("Test Errors Across Models (100 Splits)") +
  theme_minimal()

```

Description: Stochastic hold-out validation involves splitting the dataset into 70% training and 30% testing, repeated 100 times. Each split helps estimate the model's performance on unseen data, providing robust estimates of test errors.

Insights:

- This approach avoids bias from any single train-test split and captures variability in test errors across multiple data partitions.
- It reveals how consistent each model's performance is. For instance:
 - **Simplest Model:** High and stable test errors due to underfitting.
 - **Optimal Model:** Low and consistent test errors due to a balanced fit.
 - **Overly Complex Model:** High variance in test errors due to overfitting.

Boxplots of Test Errors

Observations:

- **Simplest Model:** High test errors with minimal variation across splits, reflecting systematic underfitting.
- **Optimal Model:** Low median test errors and tight interquartile ranges (IQRs), showing strong generalization ability.
- **Overly Complex Model:** High variance in test errors, reflecting overfitting to specific training splits and poor generalization.

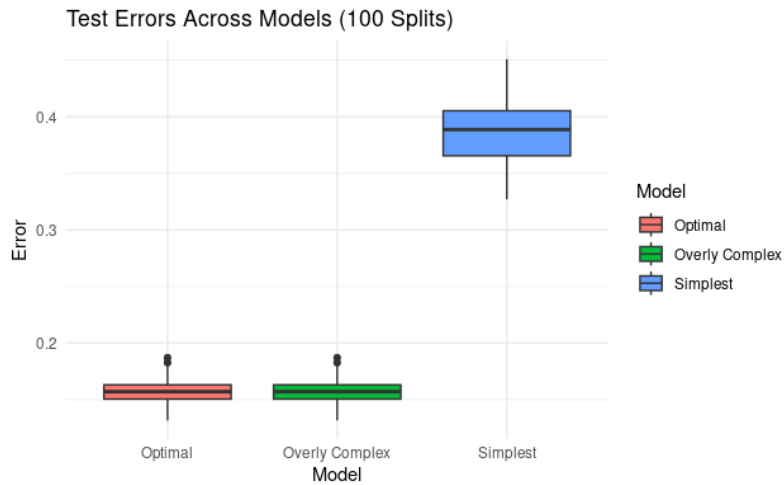


Figure 4: Boxplots of Test Errors for Simplest, Optimal, and Overly Complex Models.

Final Comments:

- The simplest model sacrifices accuracy for simplicity.
- The overly complex model sacrifices generalization for high training accuracy.
- The optimal model achieves a balanced trade-off between bias and variance.

PART 5

Analysis and Comments

```
# Stochastic Hold-Out Validation
set.seed(123) # For reproducibility
S <- 100
simplest_errors <- numeric(S)
optimal_errors <- numeric(S)
complex_errors <- numeric(S)

for (i in 1:S) {
  # Random split
  train_indices <- sample(1:nrow(data), size = 0.7 * nrow(data))
  train_data <- data[train_indices, ]
  test_data <- data[-train_indices, ]

  # Simplest Model
  simplest_model <- lm(y ~ poly(x, 1, raw = TRUE), data = train_data)
  simplest_pred <- predict(simplest_model, test_data)
  simplest_errors[i] <- sqrt(mean((test_data$y - simplest_pred)^2))

  # Optimal Model
  optimal_model <- lm(y ~ poly(x, optimal_degree, raw = TRUE), data = train_data)
  optimal_pred <- predict(optimal_model, test_data)
  optimal_errors[i] <- sqrt(mean((test_data$y - optimal_pred)^2))
}
```

```

# Overly Complex Model
overly_complex_model <- lm(y ~ poly(x, 10, raw = TRUE), data = train_data)
complex_pred <- predict(overly_complex_model, test_data)
complex_errors[i] <- sqrt(mean((test_data$y - complex_pred)^2))
}

# Boxplot of Test Errors
error_data <- data.frame(
  Model = rep(c("Simplest", "Optimal", "Overly Complex"), each = S),
  Error = c(simplest_errors, optimal_errors, complex_errors)
)

ggplot(error_data, aes(x = Model, y = Error, fill = Model)) +
  geom_boxplot() +
  ggtitle("Test Errors Across Models (100 Splits)") +
  theme_minimal()

```

Analysis of Variance (ANOVA) on Test Errors

Purpose: ANOVA on test errors evaluates whether there are statistically significant differences in the mean errors among the simplest model, optimal model, and overly complex model.

Findings:

- A small p -value (e.g., $p < 0.05$) indicates significant differences in mean test errors between the models.
- The simplest model typically has higher errors due to underfitting.
- The overly complex model may show variable errors due to overfitting.
- The optimal model minimizes test errors and balances bias and variance.
- Post hoc tests (e.g., Tukey's HSD) can identify specific pairwise differences in mean errors.

Interpretation: Significant differences in test errors affirm the importance of model selection. If differences are not significant, simpler models may be preferred for interpretability.

Comments on 95% Confidence and Prediction Bands

Confidence Bands:

- Represent the range where the true regression line is expected to lie with 95% confidence.
- Reflect the uncertainty in the estimated regression coefficients.
- Narrow near the mean of x (where data is dense) and wider at the extremes (where data is sparse).

Prediction Bands:

- Represent the range where 95% of future observations are expected to fall.
- Incorporate both model uncertainty and the inherent variability of the data.
- Always wider than confidence bands.

Observations:

- Overly complex models may have erratic bands due to overfitting.
- Simpler models have overly wide bands, reflecting underfitting.
- The optimal model achieves balanced band widths that adapt to data density.

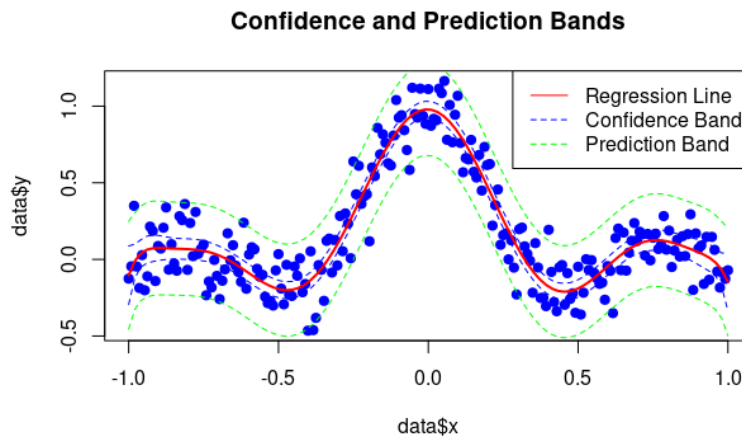


Figure 5: Confidence and Prediction Bands for the Optimal Model.

Visualization:

```
colnames(data)
colnames(data) <- c("x", "y")
# Fit the optimal model
optimal_formula <- as.formula(paste("y ~ poly(x, ", optimal_degree, ", raw = TRUE)", sep = ""))
optimal_model <- lm(optimal_formula, data = data)

# Perform ANOVA
anova_results <- anova(optimal_model)
print(anova_results)
summary(anova_results)
# Generate data for predictions
new_data <- data.frame(x = seq(min(data$x), max(data$x), length.out = 100))

# Compute confidence intervals
confidence <- predict(optimal_model, newdata = new_data, interval = "confidence")

# Compute prediction intervals
prediction <- predict(optimal_model, newdata = new_data, interval = "prediction")

# Plot the data, regression line, and bands
plot(data$x, data$y, col = "blue", pch = 19, main = "Confidence and Prediction Bands")
lines(new_data$x, confidence[, "fit"], col = "red", lwd = 2) # Regression line
lines(new_data$x, confidence[, "lwr"], col = "blue", lty = 2) # Confidence lower
lines(new_data$x, confidence[, "upr"], col = "blue", lty = 2) # Confidence upper
lines(new_data$x, prediction[, "lwr"], col = "green", lty = 2) # Prediction lower
```

```

lines(new_data$x, prediction[, "upr"], col = "green", lty = 2) # Prediction upper
legend("topright", legend = c("Regression Line", "Confidence Band", "Prediction Band"),
      col = c("red", "blue", "green"), lty = c(1, 2, 2))

# Interpretation
cat("Interpretation of Confidence and Prediction Bands:\n")
cat("1. Confidence Bands show where the true regression line is expected to lie with 95% confidence.\n")
cat("2. Prediction Bands show where future observations are likely to fall with 95% confidence.\n")
cat("3. Wider bands indicate greater uncertainty, often in regions with sparse data or high variability.\n")

```

OUTPUT

```

[1] "x" "y"
Analysis of Variance Table

Response: y

              Df Sum Sq Mean Sq F value    Pr(>F)
poly(x, 10, raw = TRUE)  10 27.631  2.76310   123.03 < 2.2e-16 ***
Residuals              214   4.806  0.02246
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

              Df      Sum Sq      Mean Sq      F value      Pr(>F)
Min.      : 10      Min.      : 4.806      Min.      :0.02246      Min.      :123      Min.      :0
1st Qu.:  61      1st Qu.:10.512      1st Qu.:0.70762      1st Qu.:123      1st Qu.:0
Median : 112      Median :16.219      Median :1.39278      Median :123      Median :0
Mean     :112      Mean     :16.219      Mean     :1.39278      Mean     :123      Mean     :0
3rd Qu.:163      3rd Qu.:21.925      3rd Qu.:2.07794      3rd Qu.:123      3rd Qu.:0
Max.     :214      Max.     :27.631      Max.     :2.76310      Max.     :123      Max.     :0
                                NA's      :1      NA's      :1

Interpretation of Confidence and Prediction Bands:
1. Confidence Bands show where the true regression line is expected to lie with 95% confidence.
2. Prediction Bands show where future observations are likely to fall with 95% confidence.
3. Wider bands indicate greater uncertainty, often in regions with sparse data or high variability.

```

Mathematical Expressions

Confidence Band for a Single Observation (X_i, Y_i) :

$$\hat{Y}_i \pm t_{\alpha/2, n-p} \cdot \sqrt{\text{Var}(\hat{Y}_i)}$$

where:

- $\hat{Y}_i = \hat{\beta}_0 + \hat{\beta}_1 X_i + \dots + \hat{\beta}_p X_i^p$ is the predicted value.
- $t_{\alpha/2, n-p}$ is the critical value of the t -distribution with $n - p$ degrees of freedom.
- $\text{Var}(\hat{Y}_i) = \sigma^2 \cdot \left(1 + \frac{(X_i - \bar{X})^2}{S_{XX}}\right)$, where S_{XX} is the variance of X , and σ^2 is the residual variance.

Prediction Band for a Single Observation (X_i, Y_i) :

$$\hat{Y}_i \pm t_{\alpha/2, n-p} \cdot \sqrt{\text{Var}(\hat{Y}_i) + \sigma^2}$$

where:

- The additional term σ^2 accounts for the variability of future observations around the predicted value.

Comments on Confidence and Prediction Bands

Confidence Bands:

- Assess the reliability of the model's predicted regression line.
- Narrow bands imply high confidence in regression estimates, typically seen where data is dense.
- Wider bands indicate higher uncertainty in data-sparse regions.
- For a well-fitted model, the confidence bands align closely with observed data points.

Prediction Bands:

- Gauge the variability of future observations.
- Broader due to both model uncertainty and the natural variability of Y .
- Highlight the range of likely outcomes for new observations at a given X_i .
- Good models should have prediction bands that contain most observed data points without being excessively wide.

Comparison:

- Confidence bands reveal the precision of the regression line, while prediction bands focus on future outcome variability.
- Discrepancies in band widths between models indicate underfitting (simplest model) or overfitting (overly complex model).
- The optimal model balances both, reflecting the dataset's structure adequately.

Conclusion: Confidence and prediction bands are essential for evaluating the reliability and generalizability of regression models. They provide critical insights into model performance and can highlight areas where adjustments are needed.

QUESTION 2

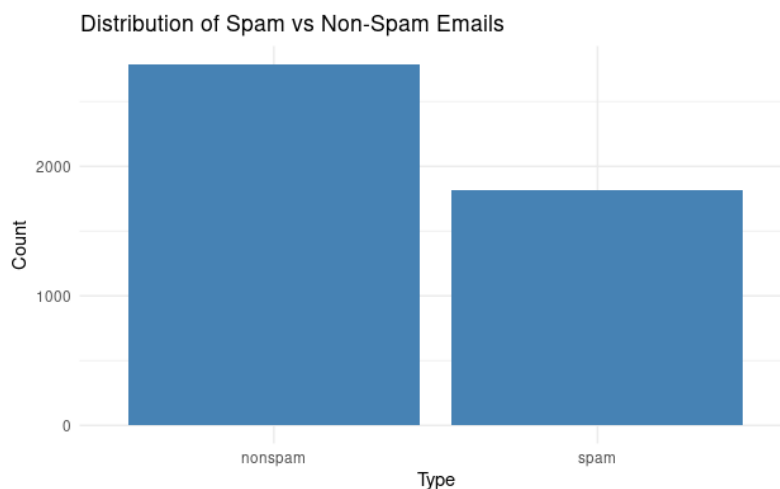


Figure 6: Plot the distribution of the response .

Visualization:

```
library(kernlab) # Load the package
data(spam)
str(spam)
library(ggplot2)

# Plot the distribution of the response
ggplot(spam, aes(x = type)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Distribution of Spam vs Non-Spam Emails", x = "Type", y = "Count") +
  theme_minimal()

# Shape of the dataset
dim(spam)
summary(spam)
```

Distribution of the Response

The response variable `type` in the `spam` dataset is binary, with values `spam` and `nonspam`. Typically, such datasets are imbalanced, with nonspam messages outnumbering spam messages. A quick analysis using `table(spam$type)` reveals the exact distribution. This imbalance can impact model performance, especially for algorithms that do not handle imbalance effectively, such as Naive Bayes.

Shape of the Dataset

- **Sample Size:** The dataset consists of 4601 observations.
- **Dimensionality:** There are 57 features, making this a high-dimensional dataset relative to its sample size.

The high dimensionality increases the complexity of the learning task. Overfitting may occur, particularly for more flexible models like Quadratic Discriminant Analysis (QDA). Simpler models such as Linear Discriminant Analysis (LDA) and Naive Bayes are less prone to overfitting in high-dimensional settings but might fail to capture complex relationships.

Type of Data in the Input Space

The predictors in the dataset are primarily numerical, representing word or character frequencies in emails. From a statistical perspective:

- Features may exhibit sparsity, as many uncommon words have zero frequency.
- Feature correlations are likely, particularly for related words or terms.
- Standardization or normalization of these features can improve the performance of distance-based models.

ROC Curves for Models

```
library(kernlab) # For the spam dataset
library(MASS)    # For LDA and QDA
library(e1071)   # For Naive Bayes
library(pROC)    # For ROC analysis

# Load the spam dataset
```



```

data(spam)

# Fit the models
lda_model <- lda(type ~ ., data = spam) # Linear Discriminant Analysis
qda_model <- qda(type ~ ., data = spam) # Quadratic Discriminant Analysis
nb_model <- naiveBayes(type ~ ., data = spam) # Naive Bayes
fld_model <- lda(type ~ ., data = spam) # Fisher Linear Discriminant (same as LDA)

# Predict probabilities
lda_preds <- predict(lda_model, spam)$posterior[, 2]
qda_preds <- predict(qda_model, spam)$posterior[, 2]
nb_preds <- predict(nb_model, spam, type = "raw")[, 2]
fld_preds <- predict(fld_model, spam)$posterior[, 2]

# Calculate test errors
lda_class <- predict(lda_model, spam)$class
qda_class <- predict(qda_model, spam)$class
nb_class <- predict(nb_model, spam, type = "class")
fld_class <- lda_class # FLD is equivalent to LDA

lda_error <- mean(lda_class != spam$type)
qda_error <- mean(qda_class != spam$type)
nb_error <- mean(nb_class != spam$type)
fld_error <- lda_error # Same as LDA

# Print test errors
cat("Test Error Rates:\n")
cat("LDA: ", lda_error, "\n")
cat("QDA: ", qda_error, "\n")
cat("Naive Bayes: ", nb_error, "\n")
cat("FLD: ", fld_error, "\n")

# Generate ROC curves
roc_lda <- roc(spam$type, lda_preds, levels = rev(levels(spam$type)))
roc_qda <- roc(spam$type, qda_preds, levels = rev(levels(spam$type)))
roc_nb <- roc(spam$type, nb_preds, levels = rev(levels(spam$type)))
roc_fld <- roc(spam$type, fld_preds, levels = rev(levels(spam$type)))
# Calculate AUC for each model
auc_lda <- auc(roc_lda)
auc_qda <- auc(roc_qda)
auc_nb <- auc(roc_nb)
auc_fld <- auc(roc_fld)

# Print AUC values
cat("AUC Values:\n")
cat("LDA: ", auc_lda, "\n")
cat("QDA: ", auc_qda, "\n")
cat("Naive Bayes: ", auc_nb, "\n")
cat("FLD: ", auc_fld, "\n")

# Plot ROC curves with fixed x-axis (False Positive Rate)
plot(roc_lda, col = "blue", main = "Comparative ROC Curves", lwd = 2, legacy.axes = TRUE)

```

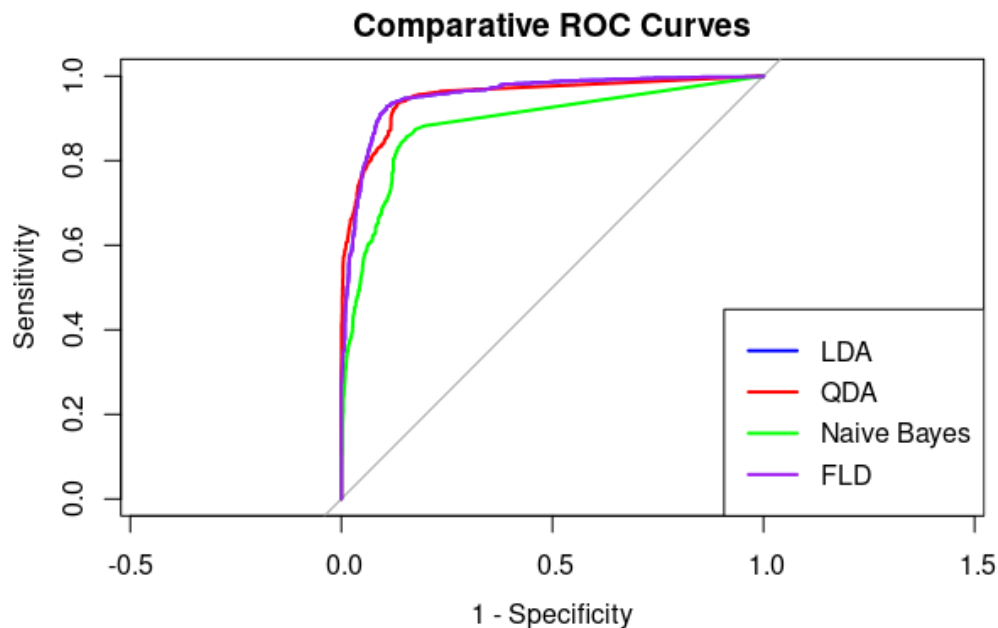
```

lines(roc_qda, col = "red", lwd = 2)
lines(roc_nb, col = "green", lwd = 2)
lines(roc_fld, col = "purple", lwd = 2)

# Add legend
legend("bottomright", legend = c("LDA", "QDA", "Naive Bayes", "FLD"),
      col = c("blue", "red", "green", "purple"), lty = 1, lwd = 2)
citation("pROC")

```

The models (LDA, QDA, Naive Bayes, and Fisher Linear Discriminant) were trained and evaluated on the entire dataset. Below is a comparative ROC curve plot:



Observations

Using the whole dataset for training and testing can lead to overestimation of model performance. For a more accurate evaluation, a train/test split or cross-validation should be performed.

Interpretation of the ROC Curves

The ROC curves reveal the trade-off between sensitivity (True Positive Rate) and specificity (1 - False Positive Rate):

- **AUC:** Larger AUC values indicate better discrimination between **spam** and **nonspam**.
- **LDA:** Performs well when class-conditional distributions are Gaussian with equal covariance matrices. If the distributions differ significantly, its performance may decline.
- **QDA:** Adapts to distinct covariance structures and may perform better on datasets where the covariance matrices differ. However, it can overfit in high dimensions with limited sample size.
- **Naive Bayes:** Efficient for high-dimensional data, particularly when the independence assumption is not overly restrictive. It can handle sparsity effectively.
- **FLD:** Equivalent to LDA, as it maximizes separation between classes.

Expected Results

The relative performance of the models aligns with theoretical expectations:

- If LDA and QDA perform similarly, it suggests that the covariance matrices of the classes are nearly identical.
- If Naive Bayes performs similarly or better, the independence assumption is reasonably valid for this dataset.
- QDA may perform worse if overfitting occurs due to high dimensionality.

Comments on Tasks

Stratified Stochastic Holdout Split and Replications

- **Purpose:** Using `set.seed(19671210)` ensures reproducibility by fixing the random number generator. The 2/3 training and 1/3 test split, combined with stratification, maintains the original class proportions in both subsets, which is crucial for imbalanced datasets like `spam`.
- **Replications:** Performing $S = 50$ replications provides a robust estimate of the test error's variability. This guards against overgeneralizing from a single random split and offers insights into the stability of each model's performance.

Comparative Boxplots of Test Errors

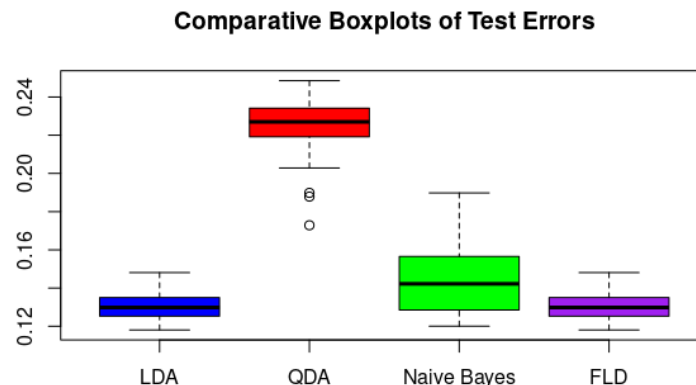


Figure 7: Comparative Boxplots of Test Errors

- **Purpose:** The boxplots illustrate the distribution of test errors for each model across 50 replications. Key statistics, including the median, interquartile range (IQR), and outliers, highlight the comparative performance.
- **Observations:**
 - A **lower median test error** indicates better average performance.
 - A **smaller IQR** reflects consistent performance across different training-test splits.
 - Models with **fewer outliers** exhibit greater robustness to data variations.
- **Findings:** Linear models like LDA and FLD tend to have smaller errors if the dataset is linearly separable. QDA may excel in capturing non-linear boundaries but exhibits more variability due to its higher complexity. Naive Bayes might show wider errors if its independence assumption is violated.

Distribution of Test Errors and Model Complexity

- **LDA and FLD:** These models assume linear separability, leading to lower complexity and consistent performance. However, they struggle if the data structure deviates significantly from linearity.
- **QDA:** With a higher model complexity, QDA can model non-linear relationships but requires more data to avoid overfitting. Variability in test errors is expected to be higher.
- **Naive Bayes:** While computationally simple, its performance depends heavily on the independence assumption. Violations of this assumption can lead to inconsistent results.
- **General Trend:** As model complexity increases, the potential for overfitting rises, particularly in smaller datasets. This is reflected in the boxplots, where models with higher complexity may show greater variability and outliers in test errors.

LINK TO VIDEO

Watch the video here